

Lab 4: Distributed Transactions

Two-Phase Commit (2PC) and Three-Phase Commit (3PC)

Distributed Computing Course

January 29, 2026

1 Introduction

This report documents the implementation of a distributed transaction system supporting both Two-Phase Commit (2PC) and Three-Phase Commit (3PC) protocols. The system consists of a coordinator node and multiple participant nodes (resource managers) that coordinate to ensure atomicity of distributed transactions across multiple nodes.

1.1 Objectives

The main objectives of this lab were to:

- Understand the atomicity problem in distributed transactions
- Implement the Two-Phase Commit (2PC) protocol
- Implement the Three-Phase Commit (3PC) protocol
- Observe blocking behavior and failure scenarios
- Compare 2PC and 3PC in terms of fault tolerance
- Deploy and evaluate transaction protocols on a multi-node system

2 System Architecture

2.1 Components

The system consists of three main components:

1. **Coordinator:** Initiates transactions, collects votes, and makes commit/abort decisions
2. **Participants:** Resource managers that validate operations locally and vote on transactions
3. **Client:** Triggers transactions by sending requests to the coordinator

2.2 Communication Model

- **Protocol:** HTTP-based message passing
- **Network:** TCP/IP communication between nodes
- **Failure Model:** Crash-stop failures (nodes fail by crashing)
- **Storage:** In-memory state with Write-Ahead Log (WAL) for durability

2.3 Transaction Model

Each transaction has:

- Unique transaction ID (e.g., TX123)
- Operation type (e.g., SET key value)
- Protocol type (2PC or 3PC)

Transactions must maintain atomicity: either all participants commit or all abort.

3 Two-Phase Commit (2PC) Implementation

3.1 Protocol Phases

The 2PC protocol consists of two phases:

3.1.1 Phase 1: Prepare

1. Coordinator sends **PREPARE** message to all participants
2. Each participant validates the operation locally
3. Participants reply with **VOTE-YES** or **VOTE-NO**
4. Participants enter **READY** state if they voted YES

3.1.2 Phase 2: Commit/Abort

1. Coordinator collects all votes
2. If all votes are YES: Coordinator sends **GLOBAL-COMMIT**
3. If any vote is NO: Coordinator sends **GLOBAL-ABORT**
4. Participants apply the decision and update their state

3.2 State Transitions

Participants maintain the following states:

- INIT: Initial state
- READY: Voted YES, waiting for coordinator decision
- COMMITTED: Transaction committed
- ABORTED: Transaction aborted

3.3 Implementation Details

The coordinator logs all transaction phases:

Listing 1: 2PC Coordinator Logging

```
1 print(f"[{NODE_ID}] {txid} PREPARE")
2 # ... collect votes ...
3 print(f"[{NODE_ID}] {txid} GLOBAL-{decision}")
```

Participants log their votes and decisions:

Listing 2: 2PC Participant Logging

```
1 print(f"[{NODE_ID}] {txid} VOTE-{vote}")
2 # ... receive decision ...
3 print(f"[{NODE_ID}] {txid} COMMIT") # or ABORT
```

4 Three-Phase Commit (3PC) Implementation

4.1 Protocol Phases

The 3PC protocol extends 2PC with an additional phase to avoid blocking:

4.1.1 Phase 1: CanCommit

1. Coordinator sends CAN_COMMIT message to all participants
2. Participants validate and vote YES/NO
3. Participants enter READY state if they voted YES

4.1.2 Phase 2: PreCommit

1. If all votes are YES, coordinator sends PRECOMMIT
2. Participants acknowledge and enter PRECOMMIT state
3. This phase ensures all participants know the decision before committing

4.1.3 Phase 3: DoCommit

1. Coordinator sends DOCOMMIT (commit) message
2. Participants apply the operation and enter COMMITTED state

4.2 Non-Blocking Property

The key advantage of 3PC is its non-blocking behavior:

- If coordinator crashes after PRECOMMIT, participants can safely commit
- Participants in PRECOMMIT state know that all other participants also voted YES
- This eliminates the indefinite blocking problem of 2PC

5 Features Implemented

5.1 Write-Ahead Log (WAL)

Both coordinator and participants maintain WALs for durability and recovery:

- **Participant WAL:** Logs all transaction events (PREPARE, COMMIT, ABORT, PRECOMMIT)
- **Coordinator WAL:** Logs transaction decisions and votes
- **Durability:** All WAL writes use `fsync()` to ensure data persistence

5.2 WAL Replay on Startup

Both components implement WAL replay for crash recovery:

Listing 3: WAL Replay Example

```
1 if WAL_PATH and os.path.exists(WAL_PATH):
2     print(f"[{NODE_ID}] Replaying WAL from {WAL_PATH}")
3     # Parse WAL entries and restore transaction state
4     # Reapply committed operations
5     print(f"[{NODE_ID}] WAL replay complete.")
```

5.3 Retry Logic

The coordinator implements retry logic with exponential backoff:

Listing 4: Retry Mechanism

```
1 def retry_post_json(url, payload, max_retries=3):
2     for attempt in range(max_retries):
3         try:
4             _, resp = post_json(url, payload)
5             return True, resp
6         except Exception:
```

```

7         if attempt < max_retries - 1:
8             time.sleep(0.5 * (attempt + 1)) # Exponential
9             backoff
        return False, None

```

This ensures decision propagation even in the presence of transient network failures.

6 Failure Scenarios

6.1 Scenario 1: Coordinator Crash in 2PC (Blocking)

Setup:

1. Coordinator sends PREPARE to all participants
2. All participants vote YES and enter READY state
3. Coordinator crashes before sending GLOBAL-COMMIT

Result:

- Participants remain in READY state indefinitely
- They cannot commit (no coordinator decision)
- They cannot abort (might have been a commit decision)
- **This demonstrates the blocking problem of 2PC**

6.2 Scenario 2: Participant Crash Before Voting

Setup:

1. Coordinator sends PREPARE
2. One participant crashes before responding
3. Coordinator times out waiting for the vote

Result:

- Coordinator receives timeout (treated as NO vote)
- Coordinator decides ABORT
- Transaction is aborted on all remaining participants
- **This demonstrates timeout handling**

6.3 Scenario 3: Coordinator Crash in 3PC (Non-Blocking)

Setup:

1. Coordinator sends CAN_COMMIT, all vote YES
2. Coordinator sends PRECOMMIT to all participants
3. Participants enter PRECOMMIT state
4. Coordinator crashes before sending DOCOMMIT

Result:

- Participants in PRECOMMIT state know all others also voted YES
- They can safely commit without waiting for coordinator
- This demonstrates the non-blocking property of 3PC

7 Comparison: 2PC vs 3PC

Property	2PC	3PC
Number of Phases	2	3
Blocking on Coordinator Failure	Yes	No
Messages per Transaction	$2n$	$3n$
Latency	Lower	Higher
Fault Tolerance	Lower	Higher
Complexity	Simpler	More Complex

Table 1: Comparison of 2PC and 3PC Protocols

7.1 Key Differences

1. **Blocking Behavior:** 2PC blocks indefinitely if coordinator fails after PREPARE. 3PC avoids this with the PRECOMMIT phase.
2. **Message Overhead:** 3PC requires one additional round of messages, increasing latency but improving fault tolerance.
3. **State Space:** 3PC introduces the PRECOMMIT state, allowing participants to know the global decision before committing.
4. **Recovery:** In 3PC, participants in PRECOMMIT can autonomously commit, while 2PC participants in READY must wait indefinitely.

8 Logging and Demonstration

8.1 Example Transaction Flow (2PC)

```
[COORD] TX123 PREPARE
[Node B] TX123 VOTE-YES
[Node C] TX123 VOTE-YES
[COORD] TX123 GLOBAL-COMMIT
[Node B] TX123 COMMIT
[Node C] TX123 COMMIT
```

8.2 Example Transaction Flow (3PC)

```
[COORD] TX456 CAN_COMMIT
[Node B] TX456 CAN_COMMIT VOTE-YES
[Node C] TX456 CAN_COMMIT VOTE-YES
[COORD] TX456 PRECOMMIT
[Node B] TX456 PRECOMMIT
[Node C] TX456 PRECOMMIT
[COORD] TX456 DOCOMMIT
[Node B] TX456 COMMIT
[Node C] TX456 COMMIT
```

9 Deployment Instructions

9.1 Starting Participants

```
1 python3 participant.py --id B --port 8001 --wal /tmp/
   participant_B.wal
2 python3 participant.py --id C --port 8002 --wal /tmp/
   participant_C.wal
3 python3 participant.py --id D --port 8003 --wal /tmp/
   participant_D.wal
```

9.2 Starting Coordinator

```
1 python3 coordinator.py --id COORD --port 8000 \
2   --participants http://<IP-B>:8001,http://<IP-C>:8002,http://<
   IP-D>:8003 \
3   --wal /tmp/coordinator.wal
```

9.3 Initiating Transactions

```
1 # 2PC Transaction
2 python3 client.py --coord http://<COORD-IP>:8000 start TX1 2PC
   SET x 5
3
```

```
4 # 3PC Transaction
5 python3 client.py --coord http://<COORD-IP>:8000 start TX2 3PC
  SET y 9
```

10 Conclusion

This lab successfully implemented both 2PC and 3PC protocols for distributed transactions. Key achievements include:

- Complete implementation of 2PC with proper state management
- Complete implementation of 3PC with non-blocking behavior
- WAL-based durability and recovery mechanisms
- Retry logic for fault tolerance
- Comprehensive logging for demonstration
- Clear demonstration of blocking vs non-blocking behavior

The implementation demonstrates the trade-offs between 2PC and 3PC: while 2PC is simpler and has lower latency, 3PC provides better fault tolerance by avoiding indefinite blocking. The choice between protocols depends on the specific requirements of the distributed system, particularly the tolerance for blocking behavior versus the need for higher message overhead.

11 References

- Distributed Computing Course Materials
- Lab 4 Instructions: Distributed Transactions (2PC / 3PC)
- Skeen, D. (1981). "Nonblocking commit protocols." ACM SIGMOD Record, 11(2), 133-142.
- Gray, J. (1978). "Notes on database operating systems." Operating Systems: An Advanced Course.